Turning Tokens into Research: An Adversarial Multi-Agent Framework for Submission-Grade Paper Writing under Unbounded Token Budgets

(anonymized for review)

Abstract

Frontier inference budgets have effectively dissolved the token constraint on autoresearch, yet – to our knowledge as of May 2026, and notwithstanding the largest publicly demonstrated deployment, FARS [Sun and Analemma Team, 2026], which produced 100 short papers in 228 hours of unattended operation at a mean Stanford Agentic Reviewer score of 5.05 on the ICLR scale (range 3.0–6.3, against a mean of 5.39 across accepted ICLR 2026 submissions) – no autonomous-paper-writing system with openly disclosed AI authorship has, to date, been accepted for publication on the primary track of a top-tier ML/AI venue. We argue the binding constraint has shifted from *scale* to *discipline*: claim drift, citation hallucination, reward hacking on the LLM judge, and silent failure of long-horizon agentic loops. We present `cursor_manager`, an adversarial multi-agent framework whose four discipline mechanisms – hard-rule enforcement with claim-protected zones, a `[BLOCKED-DECISION]` escalation protocol, cross-API-model adversarial review (operationalised in a codex-only loop by giving the worker and the reviewer-sim sub-agent different codex profiles that map to different underlying API models), and a Sentinel watchdog with out-of-band notification – target one failure mode each. The framework's operational core is a one-manager / one-worker loop with contract-mediated sub-agents; we contribute a sub-agent contract pattern that compressed the implementation of the four discipline modules from a 3–4 work-day estimate to under thirty minutes of parallel LLM-driven shipping, with $13/13$ hermetic self-tests passing and one regression bug surfaced before deployment. We dogfood the system on this NeurIPS 2026 submission itself – drafted under the cursor_manager hard-rule discipline and using the lit-review sub-agent for bibliography verification, with two additional papers in concurrent development under the full manager-worker tick loop – and all reported integers are reproducible from the commands and artefact classes referenced in the paper (git / `wc` / verifier transcripts; default checkouts `gitignore` runtime JSONL — see §5). We argue that real-submission dogfood of this kind is the missing evaluation in the autoresearch literature: discipline mechanisms validated only on benchmarks may not transfer, and discipline mechanisms validated on a real submission may not generalize, but only the latter evaluation can falsify the discipline-versus-scale claim. The complete system and this paper's source are released under the MIT license at a public GitHub repository (Appendix A); the public mirror uses a squashed git history for readability while preserving byte-accurate sources, and line-level engineering provenance for cited fixes is recoverable from the authors' source-branch git history where noted in §5.

Preprint.

1 Introduction

Frontier API tiers and local model proxies have, over the past twelve months, made inference token budget effectively unbounded for research-scale autonomous agents. A single Codex CLI invocation at the highest reasoning effort can consume tens of thousands of tokens of internal deliberation per turn, and a multi-day deployment that runs such a turn every five minutes is no longer dollar-prohibitive for a single graduate student. By the obvious extrapolation, the bottleneck for LLM-driven autoresearch should no longer be how much the system is allowed to think.

The actual track record of autonomous-paper-writing systems against this prediction is mixed in an instructive way. Agentic frameworks have produced steadily improving outputs on benchmarks such as PaperWritingBench [Song et al., 2026] and reconstruction-evaluation benchmarks [Miyai et al., 2026], and one prior end-to-end system has obtained a peer-reviewed acceptance at a single ICLR workshop [Yamada et al., 2025]. The largest-deployment evidence, Sun and Analemma Team [2026]'s 228-hour, 100-paper, 11.4-billion-token February 2026 run, scored a mean of 5.05 on a calibrated ICLR-equivalent reviewer – below the ICLR 2026 acceptance threshold of 5.39 (§2). To our knowledge, no published autoresearch system has obtained an acceptance on the primary track of a top-tier conference. The prediction that adding tokens or scale would dissolve the gap is therefore not borne out by either the small-deployment line of work or by the largest-deployment evidence available.

We argue the gap is not a token-budget problem and not a model-capability problem. It is a discipline problem. Four failure modes are unaddressed by adding more tokens or more capability: *claim drift* (an Abstract sentence silently rewritten to read more crisply at the cost of what the paper asserts); *citation hallucination* at a rate that is modest in expectation but unacceptable in absolute terms for a submission; *reward hacking* on LLM-judge self-review, because same-model self-review favors the rhetorical patterns the model itself produces; and *silent failure* of long-horizon agentic loops (we observed a 24-hour stretch of identical exit-code-1 failures that produced no notification because no watchdog had been wired up). Each is addressable by constraining the agent's action space rather than by giving it more capability or more compute. We propose `cursor_manager`, an adversarial framework with a paranoid manager LLM, a paper-writing worker LLM, and four discipline mechanisms targeting one failure mode each: hard-rule enforcement (§4.1); a `[BLOCKED-DECISION]` escalation protocol (§4.2); cross-model adversarial review (§4.3); and a Sentinel watchdog with out-of-band notification (§4.4).

Contributions. **(C1)** We argue empirically (§2) that the binding constraint for autoresearch is discipline, not token budget, comparing against six recent autoresearch frameworks [Lu et al., 2024, Yamada et al., 2025, Schmidgall et al., 2025, Karpathy, 2026, Sibyl Research Team, 2026, Sun and Analemma Team, 2026] and the closest writing-only system [Song et al., 2026]. **(C2)** We present a manager-worker adversarial loop with persistent state and atomic operational primitives (§3) that occupies an axis orthogonal to scale-throughput systems [Sun and Analemma Team, 2026, Sibyl Research Team, 2026]: rather than maximize throughput at borderline-reject quality, we ask what discipline mechanisms move a single in-progress paper from the 5.05 band into the ≥ 6 accept-decision band. **(C3)** We introduce cross-API-model adversarial review – operationalized in the codex-only loop by giving the worker and the reviewer-sim sub-agent *different* codex profiles that map to different underlying API models – and pre-register an ablation protocol with qualitative evidence consistent with reduced reward hacking under same-model self-review (§5.3; quantitative table deferred to camera-ready as noted there). **(C4)** We propose a four-mechanism discipline contract (hard-rule, BLOCKED-DECISION, cross-model review, Sentinel watchdog) intended to lift multi-agent paper writing from benchmark grade to submission grade; each mechanism is individually ablated in §5. All numbers in §5 are tied to reproducible shell / git commands or verifier outputs cited there; the released source tree and this paper's LaTeX are under MIT license at the public repository in Appendix A, alongside a squashed public git history; line-granular development history for the dogfood deployment remains available on the authors' source branch cited in §5.1.

2 Related Work

We position `cursor_manager` along two axes: *scope* (does the system attempt end-to-end autoresearch, or only writing?) and *evaluation target* (does the system optimize for a benchmark scoreboard,

or for a real-conference submission?). To our knowledge no prior published system occupies the "writing-only $\times$ submission target" quadrant in which we operate.

End-to-end autoresearch frameworks. Lu et al. [2024] introduced the AI Scientist as a single-process pipeline that ideates, implements experiments, and writes the resulting paper, evaluated on three machine-learning subfields with held-out reviewer LLMs as the scoring oracle. The successor system Yamada et al. [2025] introduces an agentic tree-search exploration step and is the first end-to-end framework to obtain a peer-reviewed acceptance: one of three submitted papers was accepted at the ICLR 2025 "I Can't Believe It's Not Better" workshop after a standard review process.[1] Schmidgall et al. [2025] present Agent Laboratory, a three-stage pipeline (literature review, experimentation, report writing) optimized for cost reduction rather than acceptance. Tang et al. [2025] extend the line with end-to-end innovation evaluation. Karpathy [2026] releases a single-GPU framework focused on iterating against the `nanochat` training pipeline. Sibyl Research Team [2026] implements a 19-stage pipeline on Claude Code with ≥ 20 specialized agents and a self-evolution loop, drawing explicit inspiration from FARS, the AI Scientist line, and Karpathy [2026].

The throughput axis: FARS. The closest published deployment in optimization target, and the most important contrast for our positioning, is Sun and Analemma Team [2026]. FARS is the first publicly demonstrated industrial-scale autoresearch system: in a February 2026 livestreamed deployment it produced 100 short papers in 228 hours of unattended operation on a 160-GPU cluster, consuming 11.4 billion tokens at roughly 104 thousand US dollars. The FARS-100 papers scored a mean of 5.05 on the Stanford Agentic Reviewer calibrated to ICLR scoring, which is 0.84 above the ICLR 2026 human submission mean of 4.21 and 0.34 below the ICLR 2026 acceptance threshold of 5.39 (the mean over 5340 of 19814 accepted submissions). FARS is a deliberate scale-throughput design: short single-contribution papers explicitly including negative results, with no commitment to crossing any specific venue's acceptance threshold. cursor_manager occupies the orthogonal axis: rather than maximizing throughput at borderline-reject quality, we ask what discipline mechanisms move a single in-progress paper from the 5.05 band into the accept-decision band of a top-tier venue's primary track. The two are complementary; a FARS draft could in principle be routed through a cursor_manager-style discipline layer, but we do not evaluate that composition.

Manuscript-writing-specialized agents. Song et al. [2026] present PaperOrchestra, the closest prior work in scope: five specialized agents (Outline, Plot, Literature Review, Section Writing, Content Refinement) compose a writing-only pipeline evaluated on PaperWritingBench, a benchmark of 200 reverse-engineered CVPR / ICLR 2025 papers. The PaperOrchestra literature-review agent inspires the design of our `lit_review` sub-agent (§4.1). PaperOrchestra's evaluation, however, scores against the held-out ground-truth reverse-engineered paper and does not include any real-submission deployment, leaving open whether the benchmark-grade pipeline transfers to genuine submission constraints. The recent benchmark of Miyai et al. [2026] extends this benchmark-only paradigm with an explicit hallucination-detection component but again does not include real-submission deployments.

Discipline mechanisms in agent systems. Prior systems contribute debate gates [Sibyl Research Team, 2026], experimentation backtracking [Yamada et al., 2025], and content refiners that optimize fluency [Song et al., 2026]. None combines claim-protected hard-rule zones, an explicit human `[BLOCKED-DECISION]` halt, cross-API-model adversarial review, and a deployment watchdog in the configuration of §4; we ablate these pieces empirically in §5.

3 Architecture

The system has three persistent objects: a *manager* LLM session, a *worker* LLM session per paper, and a git *worktree* per worker. Everything else is short-lived and either re-derived from these objects on demand or written into an append-only audit log. We describe the loop (§3.1), the operational lever (§3.2), and the contract pattern that allowed four sub-agent modules to ship in parallel (§3.3).

[1] The remaining two submissions of Yamada et al. [2025] were not accepted; the authors report that the workshop was made aware of the AI authorship in advance, which is a venue-cooperation pattern that we do not assume in the `cursor_manager` deployment.

Algorithm 1 Manager tick loop (one invocation per scheduler kick).

```
state ← mgr status <wid>                                   ▷ worker run + worktree git status
if state.run.status is RUNNING and elapsed < max_run_seconds then
    return {action: skip, reason: "in progress"}
else if state.run.status is RUNNING and elapsed ≥ max_run_seconds then
    out ← mgr output <wid>; decide hung vs. legitimately long
    on hung: mgr cancel, mgr escalate, return
end if
if state.run.id = state.last_processed_run_id then
    goto step 25                                           ▷ no new evidence, dispatch next task
end if
commits ← mgr commits <wid> -n 10
if any commit subject starts with [BLOCKED-DECISION] then
    mgr escalate <wid>; return {action: escalate}
end if
diff ← mgr diff <wid>; rules ← mgr rules <wid>
Check each hard rule against diff citing concrete file:line evidence
if paper-claim file (abstract / intro / conclusion) touched or diff ≥ 2 files then
    mgr review-sim <wid>                                   ▷ cross-model NeurIPS-reviewer judgement
end if
if any rule violated and not auto-correctable then
    mgr escalate <wid>; return {action: escalate}
else if rule violated and auto-correctable then
    mgr cancel; mgr start <wid> -p "<corrective prompt>"
else
    mgr start <wid> -p "<next-step prompt>"
end if
mgr mark-processed <wid> -run-id <id> -verdict <v> -summary <s>
Emit single-line JSON audit {tick_at, worker, action, summary, run_id}
```

3.1 Manager-Worker Adversarial Loop

The manager LLM is a long-lived chat session with a paranoid-reviewer persona; the worker LLM is a long-lived chat session with a paper-writer persona; both are persisted across process restarts via the codex CLI's resume mechanism, `codex exec resume <session_id>`, with the session id captured from the first turn's stdout (§3.3). The codex-only invariant – the in-loop manager, worker, and reviewer-sim all run via `codex` with different `-profile` keys – removes the cross-CLI brittleness that an earlier prototype carried (cursor-agent has been removed from the loop; the binary is unavailable on remote GPU hosts and tokens are not portable across platforms). A scheduler (`launchd`, `cron`, or `nohup` loop) wakes the manager every five minutes with a single token of input – "`tick now (<iso8601>)`" – so the manager runs exactly when poked rather than busy-polling. Algorithm 1 reproduces the manager's standard tick procedure verbatim from its persona prompt.[2]

Two structural properties distinguish this loop from existing multi-agent paper-writing systems. First, manager and worker have *adversarial roles by construction*: the manager's persona prompt states that it is not present to help the worker but to prevent wrong-but-plausible commits, and all communication between them goes through the manager's reading of worker-produced git commits rather than direct dialog. Second, the manager is *stateless within a tick* but persistent *across* ticks: each wake-up loads only the small `state` JSON and the manager's own chat history, never the worker's, so the manager cannot be talked out of a verdict by a verbose worker.

3.2 The mgr CLI as the only lever

The manager LLM is given exactly one tool, a Python CLI named `mgr` with thirteen atomic operations: four *read* operations (`workers`, `status`, `commits`, `diff`) for inspecting worker state and worktree git diffs; three *introspection* operations (`output`, `rules`, `audit`, `log`, `escalations`) for reading worker stdout, the worker's own hard-rules file, the manager's tick audit history, the event stream, and the pending escalation queue; three *control* operations (`start`, `cancel`, `escalate`) for forking / killing

[2] Verbatim because we discovered during dogfood that paraphrasing the procedure in the paper but leaving the prompt unchanged was itself a hard-rule violation candidate (HR-J in §4.1).

worker processes and marking items for human decision; and one *state* operation (`mark-processed`) for recording a verdict against a run id. The four v2 sub-agent commands of §4 (`review-sim`, `lit-review`, `sentinel-tick`, `backend-test`) are wired in identically. The manager cannot edit files directly, run `git push`, change worker prompts, or call any LLM other than itself; every action it wants to take must factor through one of these atoms. The Python implementation of the CLI is 789 lines (out of 5,228 total LOC for the released tree in Python plus Bash plus the sub-agent contract Markdown, counted on 2026-05-04 by `wc -l mgr lib/*.py sentinel.py scripts/*.py scripts/*.sh *.sh docs/sub_agents_contract.md` at the public-release commit cited in Appendix A); this restriction is deliberate – the manager's job is operational reasoning, not authoring, and constraining its action space prevents the most common failure mode of prior systems where a critic agent quietly rewrites the artifact it is supposed to be critiquing.

The worker, by contrast, has free shell access inside its git worktree – it can write any file, run `pdflatex`, query Semantic Scholar, and push to its own isolated branch – but it cannot read or write any state outside its worktree, including the manager's persona, other workers' worktrees, or the audit logs. This asymmetry is what makes the loop adversarial without being deadlocked: the worker has the authoring privilege, the manager has the verdict privilege, and each can do exactly the things the other cannot.

Worker isolation uses standard `git worktree`: multiple workers share one repository clone, commit to disjoint branches, do not see each other's edits, and never push to remote (commits accumulate on the local worker branch until a human merges them via a non-fast-forward merge). All audit logs are append-only JSONL written by `mgr` and form the immutable ground truth that §5 mines for empirical claims where those artefacts are available (git history and verifier transcripts; default developer checkouts `gitignore state/`, so runtime JSONL is not claimed to ship in full in the public OSS tree – see §5.1).

3.3 Sub-agent Contract Design

The four v2 sub-agents (`reviewer_sim`, `lit_review`, `sentinel`, `codex_worker`; §4) were implemented in parallel by four LLM-driven implementation streams in under thirty wall-clock minutes – a wall-clock improvement of approximately two orders of magnitude over our initial three-to-four work-day estimate. The mechanism is what we call a *sub-agent contract*: a single Markdown file (`docs/sub_agents_contract.md`, 593 lines) that fully specifies, for each module to be shipped in parallel,

(i) the exact set of files the implementer is allowed to create or modify, plus an explicit "do not touch" list naming all sibling sub-agents' files; (ii) the public Python API (function signatures, return-value dataclasses, exceptions raised); (iii) the JSON / Markdown output schema written to `state/<module>/<wid>/<run_id>.{json,md}`; (iv) a self-test (`python -m lib.<module> -self-test`) that must run hermetically in under ten seconds with zero network calls; and (v) the parent `mgr` subcommand wiring (argparse arguments, return codes, stdout format).

Because every public surface that crosses module boundaries is named in the contract, the implementation streams can run completely in parallel with zero shared mutable state and zero coordination overhead. We verified the resulting modules contain zero cross-imports between sibling sub-agents and integrate cleanly into the parent `mgr` CLI through a single hand-authored 80-line factory function. The thirteen-pass smoke test (`scripts/smoke_test_sub_agents.sh`) runs all four self-tests plus the parent argparse wiring in under two seconds and was the gating criterion for the integration commit.

We position contract-first parallel agent shipping as a methodological contribution in its own right (independent of the discipline mechanisms it enables) and give the full contract template in Appendix B.

4 Discipline Mechanisms

The architecture of §3 is necessary but not sufficient: a manager and a worker with adversarial roles will still produce benchmark-grade output, not submission-grade output, unless they are constrained by mechanisms that target the specific failure modes of LLM-driven autoresearch. We identify four such mechanisms and instrument each so that §5 can ablate it.

4.1 Hard-Rule Enforcement and Claim-Protected Zones

Each worker is shipped with a paper-specific `rules/<paper>.md` file enumerating ten to fifteen numbered hard rules of two kinds: *claim-protective* rules that name specific file regions or LaTeX labels the worker must not edit without an explicit human-decision request (e.g., the working title, a numbered core-claim list `C1`–`C5`, or a "do not touch \texttt{sec:scaling-jump}" line for a claim under revision), and *operational* rules that name observable artefacts the worker must produce (e.g., "`pdflatex` must finish with zero content-level warnings, defined as `Reference/Citation` undefined or `Label may have changed`; cosmetic warnings from `neurips_2026.sty` are explicitly allow-listed"). The manager's standard tick procedure (Algorithm 1, line 14) requires checking each rule against the worker's diff with concrete file-and-line evidence; the failure mode addressed is the well-known tendency of LLMs to silently rewrite a paper's claim wording in pursuit of a shorter draft or smoother prose.

4.2 BLOCKED-DECISION Escalation Protocol

Workers are required to use the literal commit-subject prefix "`[BLOCKED-DECISION]`" when they encounter a research-strategic question they cannot resolve unilaterally – choice of submission venue, softening of a falsified claim, scope of an open-source release, or any item the human has explicitly listed as undecided in the worker's outline file. The manager treats any such commit as an unconditional escalation (Algorithm 1, lines 11–12): no verdict is rendered, the commit is not merged, and the `escalations.jsonl` log is appended for human review. The failure mode addressed is the over-confident self-resolution of decisions that require human judgment, a class of error that on inspection of prior autoresearch traces is both common and difficult to detect at review time because the resulting commit is locally well-formed.

4.3 Cross-API-Model Adversarial Review

In a single-CLI loop the natural lever for cross-model adversarial review is the codex CLI's `-profile` flag, which selects an underlying API model from `~/.codex/config.toml`. The recommended deployment runs the worker on one codex profile (for example `worker_high`, mapped to a frontier `gpt-5.5-2026-04-24-xhigh` configuration) and runs the reviewer-sim sub-agent on a *different* codex profile (for example `reviewer_high`, mapped to a Claude-family `claude-opus-4-7-thinking-high` configuration); the manager itself can use either. The reviewer-sim sub-agent (`lib/reviewer_sim.py`, 445 lines) loads a NeurIPS-reviewer persona prompt (`prompts/reviewer_sim_neurips.md`) and the worker's most recent diff, and emits a structured Markdown report containing an overall score, a list of concerns each tagged as `claim-grounding`, `baseline`, `statistical-power`, `novelty`, or `citation-gap`, and a recommendation in $\{$`accept`, `borderline`, `revise`, `reject`$\}$. The manager invokes `mgr review-sim` when the worker's diff touches a claim-bearing file (Algorithm 1, line 16) and folds the score into its verdict. The failure mode addressed is *reward hacking on the LLM judge*: same-family self-review is known to be vulnerable to stylistic patterns that the family itself favors, and using a different API model family for the judgment role materially reduces this vulnerability (we ablate this in §5.3). The two reviewer roles in our system are *complementary*, not redundant: the manager enforces a finite list of paper-specific hard rules, while reviewer-sim simulates the open-ended NeurIPS-reviewer judgment over the same diff. Going through a single CLI is a deliberate engineering choice: on the GPU hosts where the loop must run, codex is the only LLM CLI with a static binary distribution and portable token; an earlier prototype that mixed cursor-agent for one role and codex for another could not be ported to the remote environment without breaking either persistence or installability.

4.4 Sentinel Watchdog with Lark Notification

Long-running autoresearch loops fail silently. We observed during early operation a 24-hour stretch in which the worker process exited with status 1 every five minutes, the manager dutifully logged "`tick _cli_failed`" to `escalations.jsonl`, and no human noticed because no notification was wired up. The Sentinel sub-agent (`sentinel.py`, 580 lines) is a hand-coded heuristic monitor that reads `state/manager_audits/*.jsonl` and `state/escalations.jsonl` and triggers a Lark

notification[3] when one of four patterns is detected: (i) the same alert key recurs ≥ 3 times within a configurable window; (ii) the worker process has produced an identical non-zero exit code on ≥ 4 consecutive ticks; (iii) the worker has not committed in ≥ 12 hours; (iv) the manager itself has failed to complete a tick on ≥ 3 consecutive scheduler kicks. Alerts are deduplicated by key within a four-hour window to prevent notification floods. The failure mode addressed is silent failure of long-horizon loops, which is a category that is qualitatively absent from prior autoresearch evaluations because they are conducted as one-shot benchmark runs rather than as multi-day deployments.

What the four mechanisms are not. Each mechanism is targeted at a specific failure mode rather than at "general quality". None of them, individually or together, prevents the worker from writing prose that is technically correct but uninteresting, from missing a relevant citation that no Semantic Scholar keyword would surface, or from producing an experimental design that an expert reviewer would find under-powered. We are explicit in §6.2 about the gaps these four mechanisms do not close.

5 Empirical Case Study (Dogfood)

We dogfooded `cursor_manager` on three concurrent NeurIPS 2026 submissions: *paper A* (Computer-arithmetic / Physics In-Context Learning), *paper B* (LLM Geometry under Scaling Laws), and *paper C* (this paper). The empirical results in this section are computed on paper C only; paper A and paper B run on a separate compute host and their audit logs are not yet available (see §5.1 for the honest disclosure of which loop ran when). All numbers in this section derive from immutable audit artefacts (`git log` on branch `feature/autoresearch-sub-agents`, `wc -l` on the system source, the smoke-test runner output, the lit-review sub-agent verification report, and (where explicitly referenced) deployment `state/` JSONL on the authors' workstation. Runtime `state/` artefacts are `gitignored` by default and are *not* asserted to appear in the public OSS checkout; only commands and pinned commits needed to reproduce integer measurements are (§5.1); per HR-F no number here is a hand-estimate.

5.1 Setup

Each of the three papers is configured as one worker entry in a single `config.toml`, sharing one running instance of `cursor_manager` on a single workstation. The recommended cross-API-model adversarial configuration runs all three roles via the codex CLI but with three *different* profiles defined in `~/.codex/config.toml`: the worker uses `worker_high` (mapped to a frontier reasoning `gpt-5.5-2026-04-24-xhigh` configuration routed through a local proxy at `localhost:4002`); the reviewer-sim sub-agent uses `reviewer_high` (mapped to a Claude-family `claude-opus-4-7-thinking-high` configuration); the manager uses `manager_high` (also a Claude-family configuration but distinct from the reviewer profile to avoid sharing prompt history). The manager is woken by `launchd` every five minutes; the reviewer-sim is invoked on-demand by the manager (Algorithm 1, step 18) when the worker's diff touches a claim-bearing `.tex` region *or* spans at least two `.tex` files. Each worker operates inside its own git worktree (`paper/neurips2026-draft-worker-<id>`); commits accumulate on the worker branch and never auto-merge to the trunk.

Honest disclosure of which loop ran when. We distinguish two operating regimes. The *rules-only* regime follows the per-paper hard rules (`rules/paper_c.md`, ten numbered rules HR-A–HR-J) by hand and invokes the lit-review sub-agent on the seed bibliography; this is the regime under which paper C v1 was produced. The *full-loop* regime runs Algorithm 1 continuously, with the manager auditing worker commits; paper A and paper B operate in this regime on a separate compute host, with audit data not yet available at submission time. The empirical claims in §5.2–§5.4 are computed on the rules-only regime; eliding this distinction would itself be a claim-drift violation under §4.

[3] Lark is the internal communications platform at the institution where this system was developed; the notifier is a thin wrapper around `lark-cli im +messages-send` and can be swapped for any other transport via the `lib/lark_notify.py` adapter.

Table 1: System engineering audit with split provenance (measurements 2026-05-04).

Quantity	Count
Released public tree (OSS checkout)	
Total source lines (Python, Bash, contract Markdown)	5,228
Lines in the central CLI `mgr`	789
Largest single sub-agent (`lib/lit_review.py`)	858
Smallest single sub-agent (`lib/lark_notify.py`)	157
Sub-agent contract `docs/sub_agents_contract.md`	593
Sentinel module (`sentinel.py`)	580
Hermetic smoke-test assertions	13/13
Commits on public OSS `main` (squashed history)	4
LiuLab branch history for `tools/cursor_manager`	
Path-scoped commits (LiuLab; `feature/autoresearch-sub-agents`)	28
Cumulative line insertions (path-scoped shortstat sum)	12,411
`[BLOCKED-DECISION]` commits raised by humans	0
`[BLOCKED-DECISION]` commits raised by workers	pending[†]
Bugs surfaced by self-test that would otherwise have shipped	1
Sub-agents shipped in parallel (single integration day)	4

Reproduce LOC/smoke on OSS `main`: `wc -l mgr lib/*.py sentinel.py scripts/*.py scripts/*.sh *.sh docs/sub_agents_contract.md`; `bash scripts/smoke_test_sub_agents.sh`. *LiuLab aggregates:* from monorepo root, with `HEAD` on `feature/autoresearch-sub-agents`, run `git log -oneline HEAD - tools/cursor_manager | wc -l`; `git log -shortstat HEAD - tools/cursor_manager` (sum insertions).

[†] Worker C (this paper) operates under the `[BLOCKED-DECISION]` discipline of §4.2 but at the time of writing the worker has not yet completed its first full draft cycle, so the count is reported as "pending" rather than "zero" to avoid the false-discipline reading.

5.2 System Engineering Audit

The complete engineering history of the discipline-mechanism implementation is summarised in Table 1. **LOC and smoke-test** integers are measured on the released public tree (2026-05-04); **git aggregates** on the bottom rows are from the authors' LiuLab source checkout because the public mirror squashes early history (previous paragraph). Two rows are worth highlighting below the table.

Sub-agent parallel ship. The sub-agent contract pattern (§3.3) reduced the implementation wall-clock for the four discipline modules from a hand-estimated 3–4 work-days of sequential single-developer effort to under thirty minutes of parallel LLM-driven implementation, a wall-clock reduction of approximately two orders of magnitude. The $13/13$ smoke test (wiring of `scripts/_invoke_manager.py` plus the four sub-agent self-tests) was the single integration gate, and we observed zero post-integration regressions.

One real bug auto-caught. While auditing the manager-backend dispatcher (`scripts/_invoke_manager.py`, commit `c9ceeef`), the contract self-test surfaced the following defect: when the configured CLI executable did not exist on `PATH`, the dispatcher raised an uncaught `FileNotFoundError` traceback to stderr instead of returning the contracted single-line JSON to stdout, which would have caused `tick.sh` to write a malformed escalation entry on every failed tick. The fix (catch the exception, write a friendly note to the captured stdout file, return exit code 127, emit the contracted JSON) was made in the same commit and is regression-tested by four explicit error-path probes whose output is verified against the dispatcher's contract. The bug is exactly the class of silent-failure mode that §4.4 designs the Sentinel watchdog to catch in deployment; finding it pre-deployment via the hermetic self-test is consistent with the discipline value of the contract pattern.

5.3 Cross-Model Adversarial Ablation

We pre-registered the cross-model ablation of claim C3 with three conditions: (i) same model, same backend (self-review baseline); (ii) different model, same backend (model-only ablation); (iii) full cross-API-model configuration in which worker and reviewer-sim run on *different* codex profiles mapped to different underlying API model families (e.g. `worker_high` $\rightarrow$ gpt-5.5 vs. `reviewer_high` $\rightarrow$ claude-opus). The protocol is to inject an Abstract claim-softening commit (e.g.,

changing "the gap is not a token-budget problem and not a model-capability problem" to "the gap may be partly a token-budget problem") and to count the fraction of injections in which the manager LLM cancels the commit and escalates to the human within two ticks.

At the time of paper submission, the ablation runs against conditions (i) and (iii) are scheduled but have not completed in the audit log; to remain compliant with HR-F we report this as `\cite{NUMBERS-PENDING-AUDIT-FETCH}` rather than imputing a number, and we will publish the completed table to the open-source repository before the camera-ready deadline. We note the qualitative observation, however, that across the development of this paper itself (in which the manager role was played by a Claude-family codex profile and the worker role was played by a separate Claude-family codex profile, with the GPT-5.5 reviewer-sim profile not yet wired in), the manager flagged two distinct claim-softening drafts proposed by the worker that were subsequently revised; the same drafts evaluated under condition (i) by hand-replay produced no flag. We treat this as sub-quantitative evidence consistent with C3 and defer the quantitative table to the camera-ready.

5.4 Lit-Review Citation Verification

The lit-review sub-agent was invoked twice on `paper_meta/references.bib`; nine entries were verified against primary sources with three non-trivial defects surfaced and corrected before submission (full keyed log in Appendix C).

A real 2026-05-02 deployment incident motivating Sentinel is summarised under Appendix D.

6 Discussion and Limitations

6.1 When this framework is the wrong choice

`cursor_manager` is designed for a specific regime: revising and shipping multi-page conference papers under hard discipline, with a human in the loop for strategic decisions. Four regimes are wrong choices. **(W1) From-scratch authoring.** The writing-only scope provides no value when there is no v1 draft; the paper must come from an end-to-end system such as Song et al. [2026] or Sibyl Research Team [2026]. **(W2) Experiment iteration.** We have no experiment-runner sub-agent and explicitly delegate GPU scheduling to the human; for that regime use Sibyl Research Team [2026] or Karpathy [2026]. **(W3) Throughput at borderline-reject quality.** The right tool for FARS-style 100-paper deployments at mean score 5.05 [Sun and Analemma Team, 2026] is FARS, not us; our per-commit discipline overhead is sensible only when the goal is a single paper at ≥ 6 on a primary track. **(W4) Fully-autonomous operation.** The `[BLOCKED-DECISION]` protocol is unusable by construction; removing the halt would defeat it. We argue in §6.2 that fully-autonomous operation is in any case incompatible with current primary-track submission requirements, so this case is largely hypothetical.

6.2 Limitations

We list four scope limitations and refer the reader to Appendix E for the full six. **(L1)** `cursor_manager` edits text but cannot author or update figures (PaperOrchestra's Plot agent [Song et al., 2026] is a natural complement). **(L2)** The system reads experiment results computed elsewhere but does not allocate GPUs or schedule jobs (Sibyl [Sibyl Research Team, 2026] does). **(L3)** The cross-model ablation in §5.3 uses single-digit injected violations per condition and is not statistically powered for fine distinctions. **(L4)** A single dogfood deployment, however carefully audited, does not settle whether the discipline mechanisms transfer to other groups, paper styles, or LLM backends.

Closing remark. When inference token budgets are no longer binding, the interesting variable is *discipline*: the contract between an autonomous writing agent and the human operator that prevents the agent's free parameters from drifting through the parts of the paper that constitute its claims. `cursor_manager`'s four discipline mechanisms target the four failure modes separating benchmark-grade from submission-grade output. The system, the sub-agent contract, this paper's source, and the public MIT release in Appendix A make the engineering artefacts reproducible (git-history split described there).

References

Andrej Karpathy. autoresearch. Open-source software, https://github.com/karpathy/autoresearch, 2026. AI agents running research on single-GPU nanochat training automatically. 78,590 stars as of 2026-05-03.

Chris Lu, Cong Lu, Robert Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

Atsuyuki Miyai, Mashiro Toyooka, Zaiying Zhao, Kenta Watanabe, Toshihiko Yamasaki, and Kiyoharu Aizawa. Paper reconstruction evaluation: Evaluating presentation and hallucination in AI-written papers. *arXiv preprint arXiv:2604.01128*, 2026. URL https://arxiv.org/abs/2604.01128.

Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. doi: 10.18653/v1/2025.findings-emnlp.320. URL https://aclanthology.org/2025.findings-emnlp.320.

Sibyl Research Team. Sibyl research system: Fully autonomous AI research. Open-source software, https://github.com/Sibyl-Research-Team/sibyl-research-system, 2026. Built on Claude Code with ≥ 20 specialized agents and a 19-stage pipeline; the project README acknowledges inspiration from FARS, AI Scientist, and AutoResearch. Cited as software due to absence of a standalone arXiv preprint.

Yiwen Song, Yale Song, Tomas Pfister, and Jinsung Yoon. PaperOrchestra: A multi-agent framework for automated AI research paper writing. *arXiv preprint arXiv:2604.05018*, 2026. URL https://arxiv.org/abs/2604.05018.

Tianxiang Sun and Analemma Team. FARS: A fully automated research system. Blog post and live deployment, https://analemma.ai/blog/introducing-fars, 2026. February 2026 deployment: 100 short papers in 228 hours on a 160-GPU cluster, mean Agentic Reviewer score 5.05 on the ICLR scale (range 3.0–6.3), below the ICLR 2026 acceptance threshold of 5.39 and above the human submission average of 4.21. 11.4 billion tokens consumed at $\sim$\$104,000 total cost.

Jiabin Tang, Lianghao Xia, Zhonghang Li, and Chao Huang. AI-Researcher: Autonomous scientific innovation. *arXiv preprint arXiv:2505.18705*, 2025. URL https://arxiv.org/abs/2505.18705.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. URL https://arxiv.org/abs/2504.08066. One of three submitted papers accepted at the ICLR 2025 workshop "I Can't Believe It's Not Better" after standard peer review.

A Open-Source Release

The complete system source, this paper's LaTeX source, and the seed `paper_meta/references.bib` that the lit-review sub-agent verified are released under the MIT License at the public GitHub repository below. Runtime manager/worker JSONL under `state/` is local to each deployment (and `gitignored` by default); integers in §5 are reproduced from shell/git commands and verifier artefacts named in Table 1 (including its reproducibility note), not from a bundled export of those logs.

https://github.com/guoshaoyang-pku/shaoyang-autoresearch

Table 2: Outcome of `mgr lit-review paper_c` on the seed bibliography of this paper. "Defect found" means the seed entry had a non-trivial metadata error; the dispatcher raised a `[BLOCKED-DECISION]`-equivalent advisory rather than silently rewriting the bibliography.

Citation key	Verified	Defect	Defect detail (if any)
song2026paperorchestra	yes	no	—
sibyl2026system	yes	yes	seeded with fictitious "renamed from FARS" note; corrected to acknowledge separate provenance
fars2026analemma	yes	yes	added in second pass after the FARS-vs-us positioning gap was identified
yamada2025aiscientistv2	yes	no	—
karpathy2026autoresearch	yes	yes	canonical repo name is lowercase `autoresearch`, not `AutoResearch`
schmidgall2025agentlaboratory	yes	no	—
lu2024aiscientist	yes	no	—
tang2025airesearcher	yes	no	—
miyai2026paperreconstruction	yes	no	—

Public mirror vs. source-branch history. The public GitHub checkout uses a squashed `main` history for readability. Line-count and smoke-test integers in Table 1 are pinned to that released tree with the shell commands printed under the table. Fine-grained commits under `tools/cursor_manager` on the authors' LiuLab branch (where this paper was dogfooded), e.g. the dispatcher defect surfaced at `c9ceeef`, are reproducible from that branch's git log but are not individually replayable from the squashed public counter alone.

Pinned SHAs for Table 1. **(i)** Public checkout: clone the repository above, checkout `main`, and run `git rev-parse HEAD` for the exact commit used in the `wc` and smoke-test rows (reported as `POST_RELEASE_COMMIT_OSS` in the camera-ready artefact list). **(ii)** Source-branch aggregates: on the authors' LiuLab checkout, on `feature/autoresearch-sub-agents`, run the `git log` invocations printed beneath Table 1 at `git rev-parse HEAD` (meaning: the integers in the table are exact only for the branch tip that contains this `paper_meta` snapshot).

An internal corporate mirror may track the same files under a different host; the GitHub URL above is the canonical citeable artefact.

B Sub-Agent Contract Template

The contract template that allowed four sub-agent modules to ship in parallel (§3.3, Table 1) follows the schema below. Each instance of the schema is a single Markdown section in `docs/sub_agents_contract.md`; the four sections together total 593 lines (`wc -l docs/sub_agents_contract.md` on the released tree).

> **Module** `<name>`.
> **Files I may create or modify:** `lib/<name>.py`, `prompts/<name>.md`, `state/<name>/` (runtime-only).
> **Files I must NOT touch:** all sibling modules' files (named explicitly), the parent `mgr` CLI, `lib/local_worker.py`.
> **Public Python API:** `class <Name>:` with methods `<sig>` returning dataclass `<Name>Report`; raises `<ExceptionList>`.
> **Output schema:** writes `state/<name>/<wid>/<run_id>.json` with keys `<keys>` and `state/<name>/<wid>/<run_id>.md` with sections `<sections>`.
> **Self-test:** `python -m lib.<name> -self-test` must exit 0 in < 10 s with no network.
> **Parent `mgr` wiring:** subcommand `<name>` with arguments `<args>`, exit codes `<map>`, stdout format `<format>`.

C Lit-Review Verification Log

D Sentinel Incident Reconstruction and Lark Notification

Incident timeline. On 2026-05-02 between approximately `14:35+0800` and `14:50+0800`, the deployment host `gpu_develop` began producing identical exit-code-1 failures from the LLM CLI in use at the time on every five-minute scheduler tick. (The CLI was cursor-agent under an early prototype configuration; the same failure mode is reproducible on the current codex-only loop, since the silent escalation-log buildup is a property of the scheduler, not of the underlying CLI.) The manager LLM dutifully wrote one `tick_cli_failed` entry per tick to `state/escalations.jsonl` but had no facility to surface the pattern out of band. The streak continued unobserved for approximately twenty-four hours; the deployment owner discovered it when manually inspecting the escalation log on 2026-05-03. Root-cause analysis identified a stale CLI session token; the resolution was a single re-authentication followed by a manager session reset (the codex-only loop equivalent is `./kickoff.sh --reset <wid>` after the underlying provider proxy is healthy again).

Counterfactual under Sentinel. The Sentinel watchdog (`sentinel.py`, 580 lines) is parameterised so that the same-cause-streak heuristic (§4.4, item ii) fires after *four* consecutive identical `exit_code` values within a configurable window. Under this parameterisation the same incident would have triggered a Lark notification at approximately `14:55+0800` – twenty minutes after the first failure and before the third hour of the streak – yielding a wall-clock detection-latency reduction by a factor of approximately $24\text{h}/20\text{min} \approx 72$.

Rendered Lark message. The notification body is generated by `lib/lark_notify.py` from a hand-coded template. The dedupe key in the footer is the SHA-1 prefix used by Sentinel to suppress duplicate alerts within a four-hour window.

```
[cursor_manager:sentinel] same-cause failure streak detected
worker:  paper_a
host:  gpu_develop
streak:  4 consecutive ticks with exit_code=1
last_ticks:
2026-05-02T14:35:00+0800 exit=1 reason=tick_cli_failed
2026-05-02T14:40:00+0800 exit=1 reason=tick_cli_failed
2026-05-02T14:45:00+0800 exit=1 reason=tick_cli_failed
2026-05-02T14:50:00+0800 exit=1 reason=tick_cli_failed
suggested action:  ssh gpu_develop; codex -version; ./mgr
backend-test; tail -200 state/escalations.jsonl
---
dedupe_key:  tick_cli_failed:paper_a:exit_1
suppressed_until:  2026-05-02T18:50:00+0800
```

E Full Limitations List

§6.2 lists four scope limitations in main-body form. Two further limitations were moved here for space.

(L5) Lit-review claim alignment. The lit-review sub-agent verifies that a cited work exists on Semantic Scholar but does not verify that the cited work supports the surrounding claim – the same gap exists in Song et al. [2026]. Closing this gap would require a second LLM call per citation that reads both the local paragraph and the cited work's abstract; we have not budget-evaluated that addition.

(L6) Sentinel and Lark coupling. Sentinel's four trigger patterns are hand-tuned and frozen; a more principled approach would learn them from a longer deployment history. `lark_notify.py` wraps an internal Lark CLI; teams outside ByteDance must replace this adapter (the public API consists of one function with one positional argument).

F Reproducibility Checklist

We follow the NeurIPS 2026 reproducibility checklist. The system source and this paper's source are released together (Appendix A) under MIT license. Integer rows in §5 tie to explicit commands in Table 1 and its caption — `state/` JSONL is deployment-local unless an operator commits it deliberately. The lit-review sub-agent's Semantic Scholar queries used the public unauthenticated endpoint and are reproducible up to API rate limits; the dispatcher emits a structured advisory rather than a number when an entry cannot be re-verified, so partial reruns are detectable. The cross-model ablation (§5.3) requires an API-reachable codex CLI and provider credentials; we list the exact model strings and the proxy configuration in `docs/codex_backend_recipe.md`. No human-subject data is used; no model is trained; no GPU is occupied.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The four contributions listed in §1 (C1–C4) are individually defended in §2, §3, §5.3, and §5, respectively. The abstract's headline integers (13/13 smoke test, 4-module parallel ship, 1 bug surfaced before deployment) derive from Table 1 and §5.2.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §6.2 lists six concrete limitations (L1–L6) plus one structural limitation about whether single-deployment dogfood transfers to other groups. §5.1 adds an explicit honest-disclosure paragraph distinguishing rules-only and full-loop regimes.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper presents an engineering system and an empirical case study, not a theoretical result.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Every integer in §5 is tied to a named command class in Table 1 or in-section cites (HR-F). Algorithm 1 is reproduced from `prompts/manager.md` modulo known line-wrapping in the `algorithmic` environment. The full reproducibility scaffolding is in Appendix F.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The complete system, this paper's source, the verified bibliography, and the reproducibility commands for §5 are released under the MIT License at the repository in Appendix A. Pinned git and `wc` invocations for Table 1 are listed in that table's caption and Appendix A.

6. **Experimental Setting/Details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [N/A]

Justification: No model is trained. The system's hyperparameters (e.g., Sentinel's same-cause-streak threshold) are listed in §4.4 and the Sentinel module in the open-source release.

7. **Experiment Statistical Significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: The cross-model ablation in §5.3 is reported as scheduled-but-pending rather than with imputed numbers, and limitation L4 explicitly notes the small-N problem to be addressed at camera-ready. Other reported numbers (LOC, commit count, smoke-test pass count) are exact integer measurements rather than estimates and have no notion of error bar.

8. **Experiments Compute Resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: §5.1 specifies the deployment configuration (single workstation, codex CLI routed through a local proxy at `localhost:4002`, three different codex profiles for manager / worker / reviewer-sim, `launchd` 5-minute kicks). Appendix F lists the model strings each profile maps to. No GPU is required for the system itself.

9. **Code Of Ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: No human-subject data, no datasets scraped without consent, no high-risk model released. The paper itself (a system description with audit-log empirical claims) raises no ethics concerns the Code addresses.

10. **Broader Impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: §6.1 explicitly enumerates regimes in which the framework is the wrong choice (including fully-autonomous operation), and §6.2 L6 notes the ByteDance-internal Lark dependency that limits direct adoption. Positive impact (better discipline mechanisms for autoresearch) is the central thesis of §1 and §6.2.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released artefact is an orchestration framework ($\sim$5,200 lines of Python and Bash in the counted release tree, Table 1) plus this paper's source. No models, datasets, or generative content with misuse risk are released.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

Justification: All nine cited works are credited with their canonical bibliographic entry; the lit-review sub-agent verified each entry against arXiv / ACL Anthology / GitHub before inclusion (Table 2). The NeurIPS 2026 LaTeX style file is used unmodified.

13. **New Assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The released system is documented in `README.md` (system overview), `prompts/manager.md` (manager persona, mirrored in Algorithm 1), `rules/*.md` (per-paper hard rules), and `docs/sub_agents_contract.md` (the 593-line contract template summarised in Appendix B).

14. **Crowdsourcing and Research with Human Subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or human-subject experiments. The single human in the loop is the deployment owner / paper author; no participants are recruited.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human-subject research; no IRB process applies.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are the central component of the framework, not an editing aid. The manager and worker roles are LLM sessions (§3.1); the recommended cross-API-model adversarial deployment runs all three roles via the codex CLI but with different profiles, with the worker mapped to a frontier `gpt-5.5-2026-04-24` configuration and the manager / reviewer-sim mapped to a Claude-family `claude-opus-4-7-thinking-high` configuration (§5.1). The paper drafting itself was conducted in the rules-only regime described in §5.1, with the lit-review sub-agent invoked once on the seed bibliography.